ALGORITHM 4: Optimal-DPOR algorithm considering locks.

Initial call: $Explore(\langle\rangle, \emptyset, \langle\{\langle\rangle\}, \emptyset\rangle)$

```
1  Explore(E, Sleep, WuT)
2      if enabled(s_[E]) = ∅ then
3          foreach e, e' ∈ dom(E) such that (e ≲_E e') do
4              let E' = pre(E, e);
5              let v = (notdep(e, E).ê');
6              if sleep(E') ∩ WI_[E'](v) = ∅ then
7                  wut(E') := insert_[E'](v, wut(E'));
8          foreach e ∈ dom(E) such that op_[E](e) = lock(l) do
9              let E' = pre(E, e);
10             let w = notdep(e, E);
11             foreach p such that op_[E'.w.p](next_[E'.w](p)) = lock(l) do
12                 let v = w.p;
13                 if sleep(E') ∩ WI_[E'](v) = ∅ then
14                     wut(E') := insert_[E'](v, wut(E'));
15     else
16         if WuT ≠ ⟨{⟨⟩}, ∅⟩ then
17             wut(E) := WuT;
18         else
19             choose p ∈ enabled(s_[E]);
20             wut(E) := ⟨{p}, ∅⟩;
21         sleep(E) := Sleep;
22         while ∃p ∈ wut(E) do
23             let p = min_≺{p ∈ wut(E)};
24             let Sleep' = {q ∈ sleep(E) | E ⊢ p◇q};
25             let WuT' = subtree(wut(E), p);
26             Explore(E.p, Sleep', WuT');
27             add p to sleep(E);
28             remove all sequences of form p.w from wut(E);
```

the events that happen-after e are removed from E, resulting in the suffix w. We then construct v by adding a process p that is prepared to perform a `lock(l)` operation. Since by construction w does not contain any operations on the lock l, and since the lock could be taken after E by $\widehat{e}$, it is ensured that the lock can be taken after $E'.w$ by any process that is ready to do so. Thereafter, we perform the same test as in Algorithm 2 to decide whether the wakeup tree at E' must be extended in order to explore some sequence that is equivalent to $E'.v$.

Let us next consider how to extend the soundness proof of Algorithm 2 to cover Algorithm 4. The overall structure of the proof remains the same; below we indicate where and how the proof needs to be extended.

The first extension is needed when we get to Claim 7.6, and the event performed by q is of form `lock(l)`. In this case, also e_q must perform `lock(l)` (by another process). But it is obvious that now $E'_q \vdash q.w'_q.w_R$ does not hold, since both $next_{[E'_q]}(q)$ and e_q take the lock. We therefore